

SBA301

SLOT 9 – CHAPTER 09

REACT ROUTER AND SINGLE PAGE APPLICATION (SPA)

React Router Demo • URL State • Nested Routes • Navigation Workflow

MENTAL MODEL — URL → Router → Route Match → Layout → Page Component → UI. URL không chỉ là địa chỉ; trong SPA, URL là một phần của trạng thái điều hướng.

Lớp: SE1910

Môn: SBA301 – Integrate Single Page Application with Spring Boot

Thời lượng gợi ý: 135 phút trên lớp + 2–3 giờ tự học/thực hành

Đối tượng: sinh viên Bachelor; dùng cho giảng dạy, tự học và chuẩn bị bài thực hành/assignment.

Phiên bản tài liệu: Student & Instructor Guide

0. SINH VIÊN CẦN CHUẨN BỊ GÌ TRƯỚC KHI HỌC SLOT 9

Slot 9 không bắt đầu lại từ Router cơ bản. Sinh viên cần mang theo kiến thức về Hook, navigation và Client–Server để tập trung vào cách tổ chức một SPA hoàn chỉnh, có URL có nghĩa, route tree rõ ràng và workflow điều hướng kiểm chứng được.

0.1 Kiến thức cần ôn lại

Nguồn kiến thức	Cần nhớ	Vai trò trong Slot 9
Slot 6 – React Hook	useState, useEffect, useContext, useRef, Rules of Hooks	Router hooks vẫn tuân thủ Rules of Hooks; effect có thể phản ứng với location.
Slot 7 – Handling Navigation	BrowserRouter, Routes, Route, Link/NavLink, params, query params, nested route	Slot 9 hệ thống hóa thành kiến trúc SPA và tích hợp các kỹ thuật vào một demo hoàn chỉnh.
Slot 8 – Client–Server	HTTP, JSON, REST, endpoint, request/response	Phân biệt client route /products/1 và API endpoint /api/products/1; chuẩn bị route-driven data fetching.
JavaScript	array.map/find/filter, object, destructuring, event handler	Dùng mock data và xử lý dữ liệu theo URL.

0.2 Kỹ năng và công cụ

- Chạy được project React/Vite bằng npm run dev.
- Biết mở Terminal trong IntelliJ IDEA và đọc lỗi build/runtime.
- Biết dùng Browser DevTools: Console, Network, Elements.
- Đã cài Node phù hợp với project lớp; npm hoạt động.
- Có thể cài package bằng npm và đọc package.json.
- Biết tạo component, props, useState và map() để render danh sách.

0.3 Pre-class Learning Note 3–2–1

Trước buổi học, sinh viên nộp một Learning Note ngắn: 3 khái niệm đã hiểu, 2 thí nghiệm muốn thử, 1 câu hỏi chưa rõ về SPA/Router. Không yêu cầu câu trả lời hoàn hảo; mục tiêu là kích hoạt kiến thức nền.

0.4 Diagnostic – 6 câu tự kiểm tra

1. Vì sao dùng state currentPage để chuyển màn hình không thay thế được URL routing?
2. Link khác thẻ <a> ở điều hướng nội bộ SPA như thế nào?
3. Route param và query param giải quyết hai loại thông tin nào?
4. Browser Back/Forward dựa trên cơ chế nào?
5. Nếu truy cập trực tiếp /products/3 rồi refresh, vì sao deployment có thể trả 404 dù route React tồn tại?
6. React route /products/3 và REST endpoint /api/products/3 có giống nhau không?

HUMAN GATE 0 — Nếu chưa giải thích được ít nhất 4/6 câu, hãy xem lại Slot 7 trước khi bước vào phần demo.

I. TÓM TẮT CÁC CHỦ ĐỀ & MỤC TIÊU TRONG BÀI HỌC

1. Tóm tắt kiến thức Slot 8 và vấn đề Slot 9 giải quyết

Kiến thức Slot 8	Đã giải quyết	Câu hỏi chuyển tiếp sang Slot 9
Client–Server	Hiểu frontend/backend trao đổi request/response	Màn hình nào cần gọi tài nguyên nào khi URL thay đổi?
HTTP/REST	Biết method, endpoint, status code	Client route và API endpoint phối hợp ra sao?
JSON	Hiểu dữ liệu trao đổi	Dữ liệu nào thuộc URL, dữ liệu nào thuộc state/component?

DevTools/Postman	Biết quan sát giao tiếp mạng	Làm sao chứng minh navigation nội bộ không reload document?
------------------	------------------------------	---

2. Vị trí Slot 9 trong tiến trình

Giai đoạn	Trọng tâm	Kết nối với Slot 9
Trước Slot 9	Hook + navigation + HTTP/REST	Có component tương tác, hiểu URL và nguồn dữ liệu.
Slot 9	React Router + SPA	Tổ chức route tree, layout, URL state, history và demo SPA hoàn chỉnh.
Các slot Fetch/API	Fetch/Axios, loading/error/cache	Mỗi route có thể tải resource phù hợp.
Spring Boot REST	Controller/API contract	Client route gọi REST endpoint nhưng hai khái niệm không bị trộn lẫn.
Full-stack SPA	React + Spring Boot	Deep link, auth, data flow, deployment fallback và integration.

3. Bối cảnh bài học

Một React app có nhiều component chưa chắc là một SPA được tổ chức tốt. Khi số màn hình tăng, developer phải trả lời: URL nào đại diện cho màn hình? màn hình con nằm trong layout nào? dữ liệu định danh nằm trong path hay query? hành động nào dùng Link, hành động nào dùng useNavigate? back/forward có hoạt động đúng không? deep link có chia sẻ được không? Slot 9 chuyển những câu hỏi đó thành một mô hình kiến trúc nhất quán.

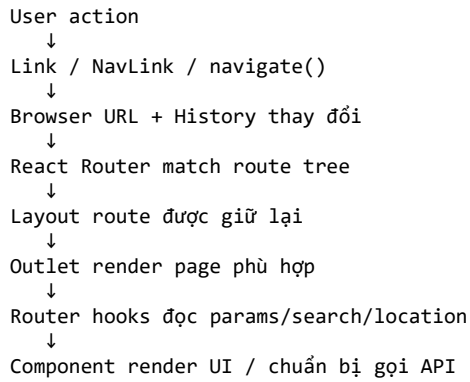
4. Mục tiêu học tập

- Giải thích được SPA, client-side routing và vai trò của URL như application state.
- Thiết kế route tree trước khi viết code.
- Cấu hình BrowserRouter, Routes, Route và layout route.
- Dùng Link/NavLink đúng mục đích; phân biệt với <a>.
- Dùng useParams, useSearchParams, useLocation và location.state đúng ngữ nghĩa.
- Dùng useNavigate và Navigate có kiểm soát.
- Tạo nested routes, index route và Outlet.
- Xử lý 404 route và resource-not-found.
- Kiểm chứng Browser Back/Forward, direct URL và refresh behavior.
- Hoàn thành React Router Demo và giải thích được route matching/data flow.

5. Khái niệm trung tâm

Khái niệm	Ý nghĩa ngắn	Ví dụ
SPA	Ứng dụng tải shell ban đầu rồi thay view phía client	/ → /products mà không reload toàn document
Client-side routing	Mapping URL với UI phía client	/products/:id → ProductDetailPage
Route tree	Cấu trúc phân cấp route/layout	MainLayout → products → :id
URL state	Phần trạng thái nên thể hiện trên URL	?category=laptop&q=air
History	Stack điều hướng của browser	Back/Forward, navigate(-1)
Deep link	URL trỏ trực tiếp tới view cụ thể	Gửi /products/3 cho bạn khác
Outlet	Vị trí render route con	MainLayout chứa <Outlet />

6. Flow tổng quát



7. React Router Demo của Slot 9

Demo chính: MiniStore SPA. Ứng dụng dùng mock products để tập trung vào routing, không biến buổi học thành bài Fetch. Các màn hình: Home, Products, Product Detail, About, Dashboard/Profile/Orders và NotFound. Filter/search của Products được lưu trên query string.

URL	View	Kỹ thuật
/	HomePage	Index/top-level route
/products	ProductsPage	List route
/products/:id	ProductDetailPage	Dynamic route + useParams
/products?q=&category=	ProductsPage	URL state + useSearchParams
/about	AboutPage	Static route
/dashboard	DashboardHome	Nested + index route
/dashboard/profile	ProfilePage	Nested route
/dashboard/orders	OrdersPage	Nested route
*	NotFoundPage	Wildcard route

8. Tư duy kiến trúc quan trọng

- Thiết kế route tree trước JSX menu.
- URL nên có nghĩa, có thể bookmark/chia sẻ và phản ánh navigation state quan trọng.
- Không lưu cùng một thông tin ở cả URL và local state nếu không có lý do; tránh hai nguồn sự thật.
- Router giải quyết navigation, không phải business data hay authorization server-side.
- Client route và REST endpoint là hai namespace khác nhau.
- Internal navigation dùng Link/NavLink; external navigation/download thường dùng <a>.
- Mọi router hook vẫn phải tuân thủ Rules of Hooks.

II. TRỌNG TÂM CỐT LỖI & KỸ NĂNG SINH VIÊN CẦN NẮM VỮNG

9. SPA vs Multi-Page Application (MPA)

Tiêu chí	SPA	MPA/traditional navigation
HTML document	Thường tải shell ban đầu	Có thể nhận document mới mỗi navigation
Chuyển view	Router phía client cập nhật UI	Server trả trang/document mới
State UI	Có thể giữ trong runtime	Dễ reset khi tải document mới
URL	Vẫn quan trọng	Quan trọng

Back/Forward	Phải tích hợp History đúng	Browser xử lý theo navigation document
Deployment	Cần fallback về index.html cho deep link	Server có route tài nguyên tương ứng

HIỂU ĐÚNG — SPA không có nghĩa là “chỉ có một URL”. Một SPA tốt vẫn có nhiều URL có nghĩa và hỗ trợ Back/Forward/deep link.

10. BrowserRouter, Routes và Route

```
<BrowserRouter>
  <Routes>
    <Route path="/" element={<HomePage />} />
    <Route path="/products" element={<ProductsPage />} />
  </Routes>
</BrowserRouter>
```

BrowserRouter kết nối React Router với browser history. Routes thực hiện route matching. Route mô tả quan hệ path → element. Ở project lớn, nên tách layout/page và giữ route definition dễ đọc.

11. Link, NavLink và <a>

Công cụ	Dùng khi	Điểm quan trọng
Link	Điều hướng nội bộ SPA	Không yêu cầu full-document navigation
NavLink	Menu cần biết active state	Có isActive để style/aria
<a>	External URL, download, protocol đặc biệt	Browser navigation bình thường; không thay Link cho internal route

```
<NavLink
  to="/products"
  className={({ isActive }) => isActive ? "nav-link active" : "nav-link"}
>
  Products
</NavLink>
```

12. Path params – useParams

Path param phù hợp với identity của resource/view. Ví dụ /products/3 xác định product id=3. Giá trị lấy từ URL là chuỗi; cần convert/validate khi so với id dạng number.

```
const { id } = useParams();
const product = products.find(p => p.id === Number(id));
```

13. Search params – useSearchParams

Query/search params phù hợp với filter, search, sort, page hoặc trạng thái muốn chia sẻ qua URL. URL trở thành single source of truth cho bộ lọc.

```
const [searchParams, setSearchParams] = useSearchParams();
const q = searchParams.get("q") ?? "";
const category = searchParams.get("category") ?? "all";

setSearchParams({ q: "air", category: "laptop" });
```

14. useLocation và navigation state

useLocation trả location hiện tại gồm pathname, search, hash và state. location.state phù hợp với thông tin điều hướng tạm thời không cần bookmark; không nên dùng thay URL cho dữ liệu cần chia sẻ/deep link.

```
const location = useLocation();
console.log(location.pathname, location.search, location.state);
```

15. useNavigate và Navigate

Công cụ	Loại	Tình huống phù hợp
useNavigate	Hook	Sau submit/login/delete; Back với navigate(-1)
Navigate	Component	Redirect trong JSX/route condition

Link/NavLink	Component	Navigation do người dùng chủ động qua UI
<pre>const navigate = useNavigate(); const handleDone = () => navigate("/products", { replace: true }); // JSX redirect <Navigate to="/products" replace /></pre>		

16. Nested route, layout route, index route và Outlet

```
<Route element={<MainLayout />}>
  <Route index element={<HomePage />} />
  <Route path="products" element={<ProductsPage />} />
  <Route path="products/:id" element={<ProductDetailPage />} />
  <Route path="dashboard" element={<DashboardLayout />}>
    <Route index element={<DashboardHome />} />
    <Route path="profile" element={<ProfilePage />} />
    <Route path="orders" element={<OrdersPage />} />
  </Route>
</Route>
```

Layout route giữ khung dùng chung. Outlet là “ổ cắm” nơi route con được render. Index route là route mặc định khi path của parent khớp nhưng chưa có child path cụ thể.

17. 404 route vs resource-not-found

Tình huống	Ví dụ	Xử lý
Route not found	/abcxyz	Wildcard path="*" → NotFoundPage
Resource not found	/products/999 nhưng route hợp lệ	ProductDetailPage không tìm được product → thông báo/resource 404 UI
API not found	GET /api/products/999 trả 404	Client xử lý HTTP response ở slot Fetch/API

18. Browser History, Back/Forward và replace

Mỗi navigation thường thêm entry vào history. navigate(-1) đi lùi một entry. replace=true thay entry hiện tại, hữu ích sau redirect hoặc login để tránh Back quay lại trang trung gian không mong muốn.

19. Deep link, refresh và deployment fallback

Trong dev server, /products/3 thường được fallback về index.html. Khi deploy SPA lên web server, server cũng phải được cấu hình để các client routes chưa phải file thật được trả về index.html. Nếu không, refresh ở /products/3 có thể nhận server 404 trước khi React Router có cơ hội chạy.

PHÂN BIỆT — Client 404 do React Router chỉ xuất hiện sau khi index.html và JavaScript được tải. Server 404 khi deep-link refresh xảy ra trước React Router.

20. Route state và state ownership

Dữ liệu	Nên ở đâu?	Ví dụ
Identity view	Path param	/products/3
Search/filter/shareable	Query param	?q=air&category=laptop
Navigation transient	location.state	{ from: "/home" }
UI local không cần URL	useState	modal open/closed
Global cross-cutting	Context khi thật sự cần	theme/user demo

21. Protected route – awareness

Protected route ở Slot 9 chỉ nên minh họa client-side UX, không được hiểu là bảo mật thật. Server vẫn phải kiểm tra authentication/authorization cho REST API.

```
function RequireAuth({ children }) {
  const isLoggedIn = false; // demo only
  return isLoggedIn ? children : <Navigate to="/login" replace />;
}
```

22. Lazy route – preview

Khi ứng dụng lớn, route-level code splitting có thể giảm bundle ban đầu. Đây là extension, không phải yêu cầu core của Slot 9.

```
const AboutPage = React.lazy(() => import("./pages/AboutPage"));

<Suspense fallback={<p>Loading...</p>}>
  <AboutPage />
</Suspense>
```

23. Kỹ năng bắt buộc

- Vẽ route tree và route matrix.
- Tạo layout route + Outlet.
- Đọc và cập nhật path/search parameters.
- Chọn đúng Link/NavLink/useNavigate/Navigate.
- Xử lý NotFound và resource not found.
- Kiểm tra Back/Forward và direct URL.
- Dùng DevTools chứng minh internal navigation không tải document mới.
- Giải thích ranh giới Router vs REST API.

24. Hiểu sai phổ biến

- SPA không cần URL.
- React Router là backend router.
- useNavigate tốt hơn Link trong mọi trường hợp.
- Query params nên đưa hết vào useState rồi bỏ URL.
- Route param luôn là number.
- Wildcard 404 xử lý luôn trường hợp product id không tồn tại.
- Client-side protected route đủ để bảo mật API.
- Refresh deep route 404 là lỗi React Router duy nhất.
- Mọi component đều cần biết location.
- Nested route chỉ là nested JSX.

III. ỨNG DỤNG THỰC TIỄN TRONG SẢN PHẨM

25. E-commerce

URL thường phản ánh product/category/search: /products, /products/123, /products?category=phone&page=2. Router tổ chức view; API cung cấp dữ liệu cho view. Cart modal có thể là local state, còn checkout step có thể là route nếu cần bookmark/history.

26. Learning Management System

Có thể dùng /courses/:courseId/lessons/:lessonId, nested layout giữ sidebar khóa học, Outlet render bài học. Query params phù hợp cho filter danh sách khóa học; route params phù hợp với identity của course/lesson.

27. Admin Dashboard

Dashboard thường có layout dùng chung và route con /dashboard/users, /dashboard/orders. Protected route giúp UX nhưng quyền thật phải được backend kiểm soát.

28. Case study: React route ↔ REST endpoint

Người dùng ở URL	React component	REST endpoint tương lai
/products	ProductsPage	GET /api/products
/products/7	ProductDetailPage	GET /api/products/7

/products?category=phone	ProductsPage	GET /api/products?category=phone
/dashboard/orders	OrdersPage	GET /api/orders

Bảng trên là bridge từ Slot 9 sang tích hợp API: URL phía client quyết định view; view quyết định endpoint cần gọi. Không đồng nhất hai lớp routing này.

IV. NỘI DUNG CHI TIẾT & REACT ROUTER DEMO

29. Demo goal và success criteria

MiniStore SPA là demo tích hợp. Sau khi hoàn thành, sinh viên phải có thể nhập URL trực tiếp, dùng menu, xem chi tiết product, thay filter trên URL, dùng Back/Forward, mở nested Dashboard và thấy 404 hợp lý.

Tiêu chí thành công	Bảng chứng
Routes chạy đúng	Ảnh Home/Products/Detail/Dashboard/404
Không full reload khi internal navigation	Network/Document không tải lại theo mỗi Link
URL state đúng	Query string thay đổi khi filter
History đúng	Back/Forward quay lại view trước
Code có layout/nested route	App.jsx + Outlet
Explainability	Route Trace Worksheet hoàn thành

30. Bước 1 – Tạo/open project và cài React Router

```
npm install
npm install react-router-dom
npm run dev
```

Trong IntelliJ: File > Open → chọn project. Mở Terminal bằng Alt+F12. Sau khi cài, kiểm tra package.json có react-router-dom và dev server chạy không lỗi.

CHECKPOINT V0 — Ứng dụng Vite chạy được trước khi thêm routing. Không debug Router trên một baseline đang lỗi.

31. Bước 2 – Cấu trúc thư mục demo

```
src/
├── components/
│   └── AppNavbar.jsx
├── data/
│   └── products.js
├── layouts/
│   ├── MainLayout.jsx
│   └── DashboardLayout.jsx
├── pages/
│   ├── HomePage.jsx
│   ├── ProductsPage.jsx
│   ├── ProductDetailPage.jsx
│   ├── AboutPage.jsx
│   ├── NotFoundPage.jsx
│   └── dashboard/
│       ├── DashboardHome.jsx
│       ├── ProfilePage.jsx
│       └── OrdersPage.jsx
├── App.jsx
├── main.jsx
└── styles.css
```

Tách pages khỏi reusable components giúp đọc route tree dễ hơn; layouts đại diện cho UI shell có Outlet.

32. Bước 3 – Mock data products.js

```
export const products = [
  { id: 1, name: "iPhone 15", category: "phone", price: 25000000 },
  { id: 2, name: "Galaxy S24", category: "phone", price: 22000000 },
  { id: 3, name: "MacBook Air M3", category: "laptop", price: 30000000 },
  { id: 4, name: "Dell XPS 13", category: "laptop", price: 35000000 },
  { id: 5, name: "iPad Air", category: "tablet", price: 18000000 }
];
```

Mock data giữ demo tập trung vào Router. Ở slot Fetch/API, mảng này sẽ được thay bằng dữ liệu từ server.

33. Bước 4 – BrowserRouter trong main.jsx

```
import React from "react";
import ReactDOM from "react-dom/client";
import { BrowserRouter } from "react-router-dom";
import App from "../App";
import "../styles.css";

ReactDOM.createRoot(document.getElementById("root")).render(
  <React.StrictMode>
    <BrowserRouter>
      <App />
    </BrowserRouter>
  </React.StrictMode>
);
```

BrowserRouter chỉ bọc một lần ở root. Nếu component dùng router hook nằm ngoài Router provider, runtime sẽ báo lỗi context.

34. Bước 5 – AppNavbar với NavLink

```
import { NavLink } from "react-router-dom";

export default function AppNavbar() {
  const cls = ({ isActive }) => isActive ? "nav-item active" : "nav-item";
  return (
    <nav className="navbar">
      <strong>MiniStore</strong>
      <NavLink to="/" end className={cls}>Home</NavLink>
      <NavLink to="/products" className={cls}>Products</NavLink>
      <NavLink to="/about" className={cls}>About</NavLink>
      <NavLink to="/dashboard" className={cls}>Dashboard</NavLink>
    </nav>
  );
}
```

Thuộc tính end giúp Home không luôn active khi URL bắt đầu bằng /.

35. Bước 6 – MainLayout với Outlet

```
import { Outlet } from "react-router-dom";
import AppNavbar from "../components/AppNavbar";

export default function MainLayout() {
  return (
    <>
      <AppNavbar />
      <main className="container"><Outlet /></main>
      <footer className="footer">SBA301 MiniStore SPA</footer>
    </>
  );
}
```

Nếu quên Outlet, URL có thể match child route nhưng page con không xuất hiện.

36. Bước 7 – HomePage và AboutPage

```
import { Link } from "react-router-dom";

export function HomePage() {
  return (
    <section>
      <h1>MiniStore SPA</h1>
      <p>React Router demo for SBA301.</p>
      <Link to="/products">Explore products</Link>
    </section>
  );
}

export function AboutPage() {
  return <section><h1>About</h1><p>Route-driven React demo.</p></section>;
}
```

37. Bước 8 – ProductsPage với useSearchParams

```
import { Link, useSearchParams } from "react-router-dom";
import { products } from "../data/products";

export default function ProductsPage() {
  const [params, setParams] = useSearchParams();
  const q = (params.get("q") ?? "").toLowerCase();
  const category = params.get("category") ?? "all";

  const filtered = products.filter(p =>
    p.name.toLowerCase().includes(q) &&
    (category === "all" || p.category === category)
  );

  const update = (key, value) => {
    const next = new URLSearchParams(params);
    value ? next.set(key, value) : next.delete(key);
    setParams(next);
  };

  return (
    <section>
      <h1>Products</h1>
      <input
        value={params.get("q") ?? ""}
        onChange={e => update("q", e.target.value)}
        placeholder="Search product"
      />
      <select
        value={category}
        onChange={e => update("category", e.target.value)}
      >
        <option value="all">All</option>
        <option value="phone">Phone</option>
        <option value="laptop">Laptop</option>
        <option value="tablet">Tablet</option>
      </select>

      <ul>
        {filtered.map(p => (
          <li key={p.id}>
            {p.name} - {p.price.toLocaleString()} VND -
            <Link to={` /products/${p.id}`}> View detail</Link>
          </li>
        ))}
      </ul>
      {filtered.length === 0 && <p>No matching products.</p>}
    </section>
  );
}
```

Điểm học tập: q/category không cần duplicate thành useState; URL là source of truth. Copy URL sau khi filter rồi mở tab mới để kiểm chứng tính shareable.

38. Bước 9 – ProductDetailPage với useParams/useNavigate

```
import { Link, useNavigate, useParams } from "react-router-dom";
import { products } from "../data/products";

export default function ProductDetailPage() {
  const { id } = useParams();
  const navigate = useNavigate();
  const product = products.find(p => p.id === Number(id));

  if (!product) {
    return (
      <section>
        <h1>Product not found</h1>
        <p>There is no product with id {id}</p>
        <Link to="/products">Back to products</Link>
      </section>
    );
  }

  return (
    <section>
      <button onClick={() => navigate(-1)}>← Back</button>
      <h1>{product.name}</h1>
      <p>Category: {product.category}</p>
      <p>Price: {product.price.toLocaleString()} VND</p>
    </section>
  );
}
```

Kiểm tra hai trường hợp khác nhau: /products/3 là route hợp lệ + resource tồn tại; /products/999 là route hợp lệ nhưng resource không tồn tại.

39. Bước 10 – DashboardLayout và nested pages

```
import { NavLink, Outlet } from "react-router-dom";

export default function DashboardLayout() {
  return (
    <section>
      <h1>Dashboard</h1>
      <div className="subnav">
        <NavLink to="." end>Overview</NavLink>
        <NavLink to="profile">Profile</NavLink>
        <NavLink to="orders">Orders</NavLink>
      </div>
      <Outlet />
    </section>
  );
}

export const DashboardHome = () => <p>Dashboard overview</p>;
export const ProfilePage = () => <p>User profile</p>;
export const OrdersPage = () => <p>Order list</p>;
```

Relative links trong nested route giảm việc hard-code path cha. Dấu . trở về route hiện tại.

40. Bước 11 – NotFoundPage

```
import { Link } from "react-router-dom";

export default function NotFoundPage() {
  return (
    <section>
      <h1>404 – Page not found</h1>
      <p>The URL does not match any client route.</p>
      <Link to="/">Go Home</Link>
    </section>
  );
}
```

41. Bước 12 – Route tree hoàn chỉnh trong App.jsx

```
import { Route, Routes, Navigate } from "react-router-dom";
import MainLayout from "../layouts/MainLayout";
import DashboardLayout, { DashboardHome, ProfilePage, OrdersPage } from "../layouts/DashboardLayout";
import { HomePage, AboutPage } from "../pages/StaticPages";
import ProductsPage from "../pages/ProductsPage";
import ProductDetailPage from "../pages/ProductDetailPage";
import NotFoundPage from "../pages/NotFoundPage";

export default function App() {
  return (
    <Routes>
      <Route element={<MainLayout />}>
        <Route index element={<HomePage />} />
        <Route path="products" element={<ProductsPage />} />
        <Route path="products/:id" element={<ProductDetailPage />} />
        <Route path="about" element={<AboutPage />} />
        <Route path="home" element={<Navigate to="/" replace />} />

        <Route path="dashboard" element={<DashboardLayout />}>
          <Route index element={<DashboardHome />} />
          <Route path="profile" element={<ProfilePage />} />
          <Route path="orders" element={<OrdersPage />} />
        </Route>

        <Route path="*" element={<NotFoundPage />} />
      </Route>
    </Routes>
  );
}
```

Điểm kiểm tra: child paths không cần dấu / khi đặt dưới parent route. Wildcard nằm trong layout để 404 vẫn giữ navbar/footer.

42. Bước 13 – CSS tối thiểu

```
body { margin: 0; font-family: Arial, sans-serif; background: #f7f8fa; }
.container { max-width: 1000px; margin: 0 auto; padding: 24px; }
.navbar { display: flex; gap: 16px; align-items: center; padding: 16px 24px; background: #17365d; }
.navbar a { color: white; text-decoration: none; }
.navbar a.active { font-weight: 700; text-decoration: underline; }
.subnav { display: flex; gap: 12px; margin-bottom: 16px; }
.subnav a.active { font-weight: 700; }
.footer { padding: 20px; text-align: center; border-top: 1px solid #ddd; }
input, select, button { margin: 4px; padding: 8px; }
```

43. Bước 14 – Verification bằng Browser DevTools

Mở DevTools > Network, chọn filter All/Doc. Bấm Home → Products → Product Detail bằng Link/NavLink. Quan sát: URL đổi, UI đổi, nhưng không có một document navigation mới tương ứng với mỗi click nội bộ. Sau đó nhấn Refresh tại /products/3 để phân biệt reload document với client navigation.

44. Bước 15 – Browser History test

Test	Thao tác	Kết quả mong đợi
T1	Home → Products → Detail → Back	Quay về Products và giữ URL/query trước đó nếu history entry còn
T2	Chọn category=laptop rồi copy URL	Tab mới mở cùng filter
T3	navigate(-1) ở Detail	Quay lại entry trước
T4	/home redirect replace	Back không mắc ở redirect trung gian theo cách không mong muốn
T5	Nhập /abcxyz	NotFoundPage trong MainLayout

45. Bước 16 – Route Trace Worksheet

URL	Matched parent/layout	Matched page	Hook/data cần đọc
-----	-----------------------	--------------	-------------------

/	MainLayout	HomePage	Không
/products?q=air	MainLayout	ProductsPage	useSearchParams: q
/products/3	MainLayout	ProductDetailPage	useParams: id=3
/dashboard/orders	MainLayout → DashboardLayout	OrdersPage	Nested matching
/nothing	MainLayout	NotFoundPage	Wildcard

46. Bước 17 – Client Route vs API Endpoint Trace

```

Browser URL:    /products/3
React matches:  ProductDetailPage
Route param:    id = "3"
Future API:     GET /api/products/3
Response JSON:  { id: 3, ... }
React renders:  product detail UI

```

Đây là workflow quan trọng để nối Slot 9 với Client–Server/Fetch: URL quyết định view; view đọc param; view gọi endpoint; server trả JSON; component render trạng thái.

47. Bước 18 – Git checkpoint

```

git status
git add .
git commit -m "Build React Router SPA demo with nested routes and URL state"
git log -1

```

CHECKPOINT V9 — Demo chỉ được xem là hoàn thành khi route matrix, Back/Forward, direct URL, filter URL và 404 đều được kiểm chứng, không chỉ khi “npm run dev chạy”.

V. KINH NGHIỆM THỰC TIỄN, TIPS & DEBUGGING

48. Debugging decision tree

```

URL không render đúng?
├─ App có nằm trong BrowserRouter?
├─ Route có nằm trong Routes?
├─ path có đúng relative/absolute?
├─ Parent layout có <Outlet />?
├─ import/export component đúng?
├─ useParams tên param có khớp :id?
├─ id string đã convert khi so với number?
├─ query param có null/default handling?
└─ Refresh deep link lỗi? → kiểm tra server fallback, không chỉ React code.

```

49. Bảng lỗi thường gặp

Dấu hiệu	Nguyên nhân thường gặp	Cách xử lý
useNavigate/useParams báo ngoài Router	Component không nằm dưới BrowserRouter	Bọc root đúng một lần
Nested URL đổi nhưng nội dung con không hiện	Thiếu Outlet	Thêm <Outlet /> vào layout cha
Home luôn active	NavLink / match prefix	Dùng end cho link Home
Không tìm product id=1	useParams trả string	Number(id) hoặc so sánh string nhất quán
Filter mất khi copy URL	Filter lưu useState thay vì query	Dùng useSearchParams làm source of truth
Refresh /products/3 404 khi deploy	Web server không fallback	Cấu hình rewrite SPA → index.html
Back quay về redirect page	Không replace redirect	Dùng replace khi phù hợp
Menu làm reload	Dùng <a href> nội bộ	Đổi thành Link/NavLink

50. Best practices

- Route path dùng lowercase, semantic và ổn định.
- Thiết kế route matrix trước khi code.
- Giữ page components mỏng; reusable UI ở components/.
- Dùng URL cho state cần share/bookmark.
- Không duplicate query params vào state nếu không cần.
- Giữ auth bảo mật ở server; client route guard chỉ là UX.
- Luôn có 404 route và resource-not-found state.
- Test direct URL, refresh, Back/Forward, không chỉ click menu.

51. Break-It Lab – cố ý tạo lỗi để học

1. Bỏ BrowserRouter rồi đọc lỗi.
2. Xóa Outlet khỏi MainLayout.
3. Đổi path products/:id thành products/:productId nhưng vẫn đọc id.
4. Dùng `<a href="/products">` thay Link và quan sát Network.
5. So sánh `p.id === id` mà không convert.
6. Lưu search keyword bằng `useState` và chứng minh reload/copy URL mất filter.
7. Deploy/preview với server không fallback (nếu môi trường cho phép) và thử deep-link refresh.

VI. BÀI TẬP CHỦ ĐỀ & BÀI TẬP TỔNG HỢP

52. Bài tập chủ đề

Bài 1 – Route Matrix

Thiết kế route matrix cho ứng dụng thư viện: Home, Books, Book Detail, Authors, About, 404.

Bài 2 – NavLink

Tạo menu và active state; Home dùng end.

Bài 3 – Dynamic route

Tạo `/students/:id` và xử lý id không tồn tại.

Bài 4 – Query state

Tạo `/books?q=&category=&page=` và giữ filter trong URL.

Bài 5 – useNavigate

Sau submit form giá lập, điều hướng về list; giải thích có/không dùng replace.

Bài 6 – location.state

Truyền `{from:"featured"}` khi mở detail; giải thích vì sao refresh có thể không phù hợp cho dữ liệu bền vững.

Bài 7 – Nested Dashboard

Tạo `/admin`, `/admin/users`, `/admin/settings` bằng Outlet.

Bài 8 – 404 phân tầng

Phân biệt route 404 với resource not found.

Bài 9 – History

Ghi trace 6 bước navigation và dự đoán Back/Forward.

Bài 10 – Client/API mapping

Ghép mỗi client route với REST endpoint tương lai.

Bài 11 – Deep link

Giải thích vì sao refresh route sâu cần server fallback.

Bài 12 – Router code review

Tìm tối thiểu 5 anti-patterns trong một route file do AI tạo.

53. Bài tập tổng hợp – Campus Course SPA

Xây dựng một SPA quản lý catalog khóa học bằng mock data. Yêu cầu: `/`, `/courses`, `/courses/:id`, `/courses?q=&level=`, `/about`, `/dashboard` với nested Profile/Progress, redirect `/home` → `/`, 404, NavLink active, Back/Forward đúng. Không gọi backend thật trong core.

Yêu cầu	Minh chứng
Route tree	Sơ đồ + App.jsx
Dynamic/detail	<code>/courses/:id</code> và invalid id
URL filter	Query params giữ được sau reload
Nested route	Dashboard + Outlet
History	Test Back/Forward
404	Unknown URL
Client/API bridge	Bảng <code>/courses/:id</code> ↔ GET <code>/api/courses/{id}</code>
Git	Commit + README

54. Human Verification Gate – 20 câu

- SPA có nghĩa là gì?
- URL có vai trò gì trong SPA?
- BrowserRouter làm gì?
- Routes và Route khác nhau thế nào?
- Link và NavLink khác gì?
- Khi nào dùng `<a>`?
- Path param phù hợp với dữ liệu nào?
- Query param phù hợp với dữ liệu nào?
- `useLocation` trả gì?
- `location.state` có nên dùng cho dữ liệu bookmark không?
- `useNavigate` khác `Navigate`?
- Outlet dùng để làm gì?
- Index route là gì?
- Route 404 khác resource 404?
- Tại sao `useParams` thường trả string?
- `replace` ảnh hưởng history thế nào?
- Deep link là gì?
- Vì sao refresh route sâu có thể server 404?
- Client route khác API endpoint ra sao?
- Client protected route có đủ bảo mật không?

VII. TỪ ĐIỂN THUẬT NGỮ – FOUNDATION GLOSSARY

Thuật ngữ	Định nghĩa	Ví dụ
SPA	Single Page Application; shell chính được tải một lần và view thay đổi phía client.	<code>/products</code> → <code>/products/3</code>
Client-side routing	Mapping URL với component ở browser.	React Router
BrowserRouter	Router dùng browser History API.	Bao App ở main.jsx

Routes	Bộ matcher chứa Route.	<Routes>...</Routes>
Route	Khai báo path → element/layout.	path="products/:id"
Route tree	Cấu trúc phân cấp routes.	MainLayout → DashboardLayout → Orders
Path param	Biến động trong path.	/products/:id
Search/query param	Dữ liệu sau ?.	?q=air&page=2
Link	Navigation nội bộ SPA.	to="/products"
NavLink	Link có active state.	Menu
useParams	Đọc path params.	id
useSearchParams	Đọc/cập nhật query string.	q/category
useLocation	Đọc location hiện tại.	pathname/search/state
useNavigate	Điều hướng bằng code.	navigate(-1)
Navigate	Component redirect.	<Navigate to="/" />
Outlet	Vị trí render child route.	Layout
Index route	Child mặc định của parent.	<Route index ... />
Wildcard route	Route bắt unmatched path.	path="**"
History	Stack navigation browser.	Back/Forward
Deep link	URL trực tiếp đến view sâu.	/products/3
URL state	State quan trọng được thể hiện trên URL.	filter/sort/page
Navigation state	State tạm thời kèm navigation.	location.state
Redirect	Chuyển từ URL này sang URL khác.	/home → /
Layout route	Route cung cấp UI shell dùng chung.	MainLayout
Route matching	Quá trình chọn branch phù hợp URL.	/dashboard/orders
Resource not found	Route hợp lệ nhưng entity không tồn tại.	/products/999
Server fallback	Web server trả index.html cho client route.	SPA deployment
Protected route	Guard UI route theo auth state.	RequireAuth
Code splitting	Tách bundle theo route/component.	React.lazy
API endpoint	URL server xử lý HTTP request.	GET /api/products/3

VIII. RESEARCH BASE & AI-BASED LEARNING

55. Nguyên tắc AI-0 / AI-Review

AI-0: trước khi hỏi AI, sinh viên phải ghi route tree dự đoán, lỗi quan sát được, URL hiện tại và giả thuyết nguyên nhân. AI-Review: sau khi nhận câu trả lời, sinh viên phải chạy lại, đọc DevTools, giải thích vì sao fix đúng và ghi evidence. Quy tắc: AI may generate; student must verify.

56. 10 prompt nghiên cứu kỹ thuật

1. Giải thích cách BrowserRouter dùng History API để đồng bộ URL và UI.
2. Phân tích route matching và nested route trong React Router hiện đại.
3. So sánh path params, query params, hash và location.state.
4. Giải thích vì sao URL state có thể tốt hơn duplicate local state.
5. Phân tích history push vs replace và ảnh hưởng Back/Forward.
6. Giải thích deep linking và server fallback khi deploy SPA.

7. Phân biệt route 404, resource 404 và HTTP 404.
8. Phân tích stale navigation state và các lỗi khi dựa quá nhiều vào location.state.
9. Giải thích accessibility của Link/NavLink và active navigation.
10. Phân tích route-level code splitting với React.lazy/Suspense.

57. 10 prompt nghiên cứu thực tiễn production

1. Thiết kế route tree cho e-commerce có catalog, cart, checkout, account và admin.
2. Trong project lớn nên tổ chức route config, layouts và page modules ra sao?
3. Làm sao quản lý breadcrumbs từ nested route?
4. Protected routes nên phối hợp với backend authorization như thế nào?
5. Làm sao giữ filter/sort/page trên URL để chia sẻ link?
6. Làm sao xử lý scroll restoration khi navigation?
7. Làm sao test React Router navigation với React Testing Library?
8. Làm sao xử lý analytics/page view trong SPA khi location thay đổi?
9. Làm sao cấu hình Nginx/Apache/static host cho deep-link fallback?
10. Khi nào nên dùng framework router/data router thay vì BrowserRouter declarative routing?

58. 10 prompt khám phá xu hướng

1. So sánh React Router declarative routing với data routers/framework mode.
2. So sánh React Router với TanStack Router về type-safety và data loading.
3. URL state management đang được dùng như thế nào trong React hiện đại?
4. Server Components/framework routing thay đổi vai trò client-side router như thế nào?
5. Lazy route và streaming UI ảnh hưởng UX navigation ra sao?
6. View Transitions API có thể cải thiện SPA navigation như thế nào?
7. React Router hiện đại hỗ trợ pending/navigation states ra sao?
8. Các lỗi accessibility phổ biến trong SPA navigation hiện nay là gì?
9. Cách observability theo dõi route change và web vitals trong SPA.
10. AI coding assistants thường tạo sai gì trong Router code và cách audit?

59. AI Verification Worksheet

Trước AI	Câu trả lời AI	Kiểm chứng	Kết luận
URL + lỗi + giả thuyết	Tóm tắt đề xuất của AI	Code chạy? Console? Network? History?	Accept / modify / reject + lý do
Route tree dự kiến	AI đề xuất route structure	So với yêu cầu và route matrix	Giữ cấu trúc tối giản nếu đủ

IX. SINH VIÊN CẦN TỰ KIỂM TRA GÌ SAU BUỔI HỌC

60. Checklist kiến thức

Nội dung tự kiểm tra	Đạt	Chưa đạt	Evidence
Tôi giải thích được SPA và client-side routing.	☐	☐	
Tôi phân biệt BrowserRouter/Routes/Route.	☐	☐	
Tôi dùng đúng Link và NavLink.	☐	☐	
Tôi đọc được path param bằng useParams.	☐	☐	
Tôi dùng query params làm URL state.	☐	☐	

Tôi biết khi nào dùng <code>useNavigate/Navigate</code> .	☐	☐	
Tôi tạo được nested route + Outlet + index route.	☐	☐	
Tôi phân biệt route 404 và resource not found.	☐	☐	
Tôi kiểm tra được Back/Forward.	☐	☐	
Tôi hiểu deep-link refresh và server fallback.	☐	☐	
Tôi phân biệt React route và REST endpoint.	☐	☐	
Tôi có thể trace URL → route → component → future API.	☐	☐	

61. Mức tự đánh giá

Mức	Biểu hiện
Mức 1 – Chưa đạt	Chưa tạo route tree; navigation lỗi; không giải thích được params/query/Outlet.
Mức 2 – Đạt	Tạo được routes/menu/detail/query/404; demo chạy nhưng giải thích history/deep link còn yếu.
Mức 3 – Tốt	Giải thích được URL state, nested route, history, deployment fallback và client/API mapping; evidence đầy đủ.

62. Exit Ticket – 5 câu

1. Vì sao URL là một phần của state trong SPA?
2. Khi nào nên dùng path param, khi nào query param?
3. Tại sao nested route cần Outlet?
4. Client route `/products/7` khác GET `/api/products/7` thế nào?
5. Một lỗi chỉ xuất hiện khi refresh `/products/7` trên server production gợi ý vấn đề gì?

63. Reflection sau buổi học

Viết 120–180 từ: một quyết định routing bạn đã thay đổi sau khi làm demo; evidence nào khiến bạn thay đổi; một phần bạn vẫn cần nghiên cứu trước khi tích hợp Fetch/API.

64. Homework

- Hoàn thiện MiniStore SPA hoặc chuyển route model sang project nhóm.
- Tạo route matrix cho project nhóm tối thiểu 8 routes.
- Đánh dấu route nào cần API endpoint tương lai.
- Thực hiện 3 bug injection và ghi Debug Log.
- Commit/push source + README + screenshots + Route Trace Worksheet.

PHỤ LỤC A. KẾ HOẠCH 135 PHÚT TRÊN LỚP

Thời lượng	Hoạt động	Kết quả
0–12'	Retrieval + diagnostic Slot 7/8	Kích hoạt URL/Router/HTTP mental model
12–27'	SPA vs MPA + URL state + history	Giải thích vì sao Router cần thiết
27–47'	Route tree, layouts, params/query/navigation	Vẽ route matrix
47–82'	Instructor guided MiniStore Router Demo	App chạy routes/nested/filter/detail

82–102'	Pair Break-It Lab	Debug Outlet/param/<a>/query state
102–118'	Client route ↔ API endpoint trace	Chuẩn bị integration
118–130'	Human Verification Gate + peer explanation	Chứng minh hiểu
130–135'	Exit Ticket	Chốt learning gap

PHỤ LỤC B. QUICK REFERENCE

Nhu cầu	Công cụ
Đi internal route	Link
Menu active	NavLink
Đọc /:id	useParams
Đọc/cập nhật ?q=	useSearchParams
Đọc pathname/state	useLocation
Navigate sau logic	useNavigate
Redirect trong JSX	Navigate
Render child route	Outlet
Default child	index route
Unmatched URL	path="*"
Quay lại	navigate(-1)
Không thêm history entry	replace

PHỤ LỤC C. EVIDENCE PACKAGE

- Ảnh route matrix/route tree.
- Screenshot Home, Products với query, Product Detail, Dashboard nested, 404.
- DevTools evidence chứng minh internal Link navigation không reload document.
- Back/Forward test log.
- Direct URL + refresh test.
- Client Route ↔ API Endpoint mapping.
- Git commit hash/log.
- AI log nếu có: prompt → output → verification → decision.

TÀI LIỆU THAM KHẢO GỢI Ý

- React Router Documentation – <https://reactrouter.com/>
- W3Schools React Router – dùng cho ôn tập cú pháp cơ bản.
- MDN – Single-page application (SPA), URL, History API.
- React.dev – Components, Hooks, lazy/Suspense (phần mở rộng).
- SBA301 Syllabus – Chapter 09: React Router and Single Page Application (SPA).

Kết thúc tài liệu. Sinh viên chỉ chuyển sang tích hợp dữ liệu server khi đã giải thích và kiểm chứng được route workflow của SPA.